

MCP-Backed Chat: Angular Frontend Architecture

Plan v2

Revised against the real MCP gateway backend at

C:\Users\katlego.gagoopane\Documents\2026\Claude\backend | Prepared 2026-08-13

What changed from v1: v1 assumed a generic Node.js bridge and a generic MCP setup. This version is grounded in the actual backend: an existing Python **FastAPI "MCP Gateway"** (`mcp-gateway/`) that already mounts the two real MCP servers — `docker-mcp` and `ignition-mcp` — as Streamable-HTTP sub-apps at `/docker` and `/ignition` , protected by an `X-API-Key` header. That service is left **unchanged**; a new component is added in front of it.

1. Objective

Today, Claude Code is invoked from a terminal and reaches the MCP Gateway's `/docker` and `/ignition` endpoints to control Docker and an Ignition industrial gateway. The objective is to let end users submit the same kind of queries from a web browser via an Angular frontend, while keeping the existing gateway exactly as it is.

2. What the Existing Backend Already Does

Piece	What it is	Detail
<code>mcp-gateway</code>	FastAPI app (<code>gateway/app.py</code>), run with uvicorn	Binds to <code>GATEWAY_HOST:GATEWAY_PORT</code> (default <code>127.0.0.1:8000</code>); mounts both MCP servers as sub-apps with a composed lifespan (<code>AsyncExitStack</code>)
<code>/docker</code>	<code>docker-mcp</code> Streamable-HTTP sub-app	Controls the host Docker daemon (docker-outside-of-docker socket mount) — effectively root on the host
<code>/ignition</code>	<code>ignition-mcp</code> Streamable-HTTP sub-app	Reads/writes tags, alarms, and scripts on a live Ignition industrial gateway
Auth today	Raw ASGI <code>APIKeyMiddleware</code>	Requires header <code>X-API-Key: <GATEWAY_API_KEY></code> on any path starting with <code>/docker</code> or <code>/ignition</code> ; <code>/health</code> is open; written as raw ASGI (not <code>BaseHTTPMiddleware</code>) specifically so it doesn't break the SSE streaming both sub-apps use
Deployment today	<code>docker-compose.yml</code> in <code>mcp-gateway/</code>	Publishes port 8000; mounts <code>/var/run/docker.sock</code> . Its own comments flag <code>GATEWAY_API_KEY</code> as "the only thing standing between the internet and [host root]" and say to keep it off public networks "unless you put a real auth layer in front of it"

Why this matters for the frontend design: the gateway's own documentation already anticipates that a real, user-facing auth layer needs to sit in front of it before it's exposed more broadly. A browser chat UI is exactly that broader exposure. The API key that guards `/docker` and `/ignition` must stay a server-to-server secret — it must never reach the browser, and it should not be treated as equivalent to authenticating an individual human user.

3. Recommended Approach (v2)

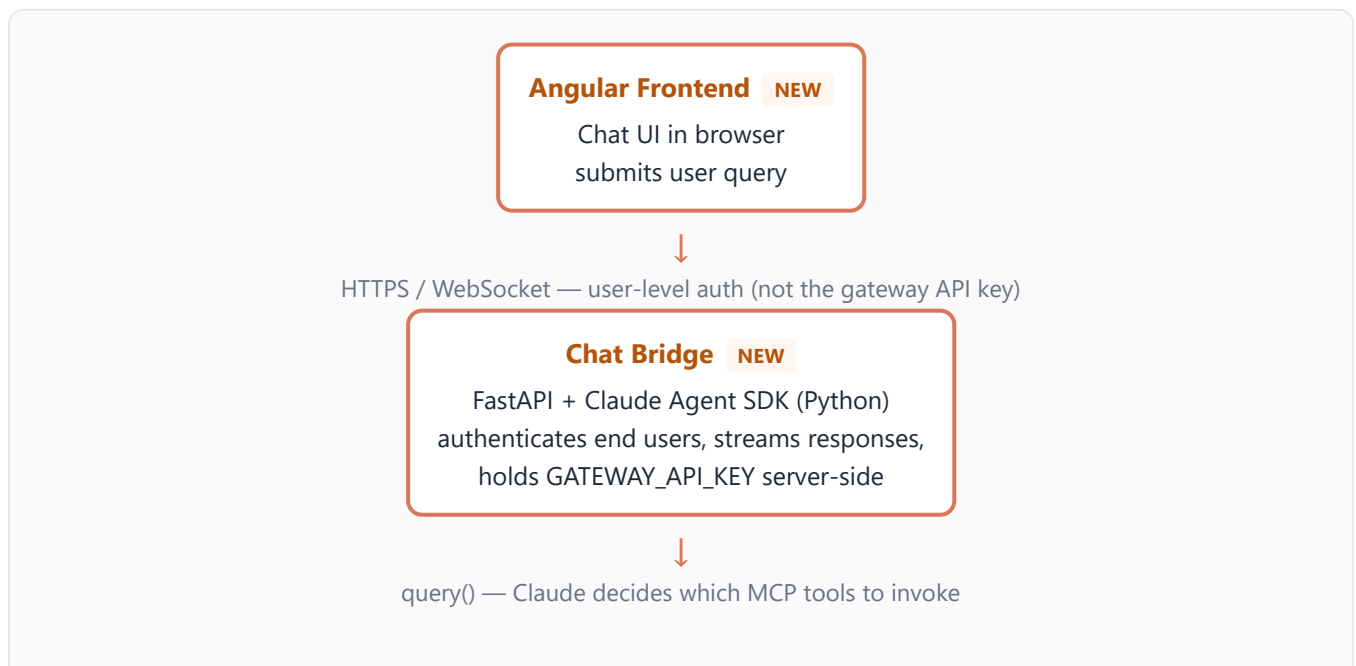
Add one new component — a **Chat Bridge** service — between the Angular app and the existing gateway. Do not modify `mcp-gateway` itself and do not expose `/docker` or `/ignition` directly to the browser or to the public internet.

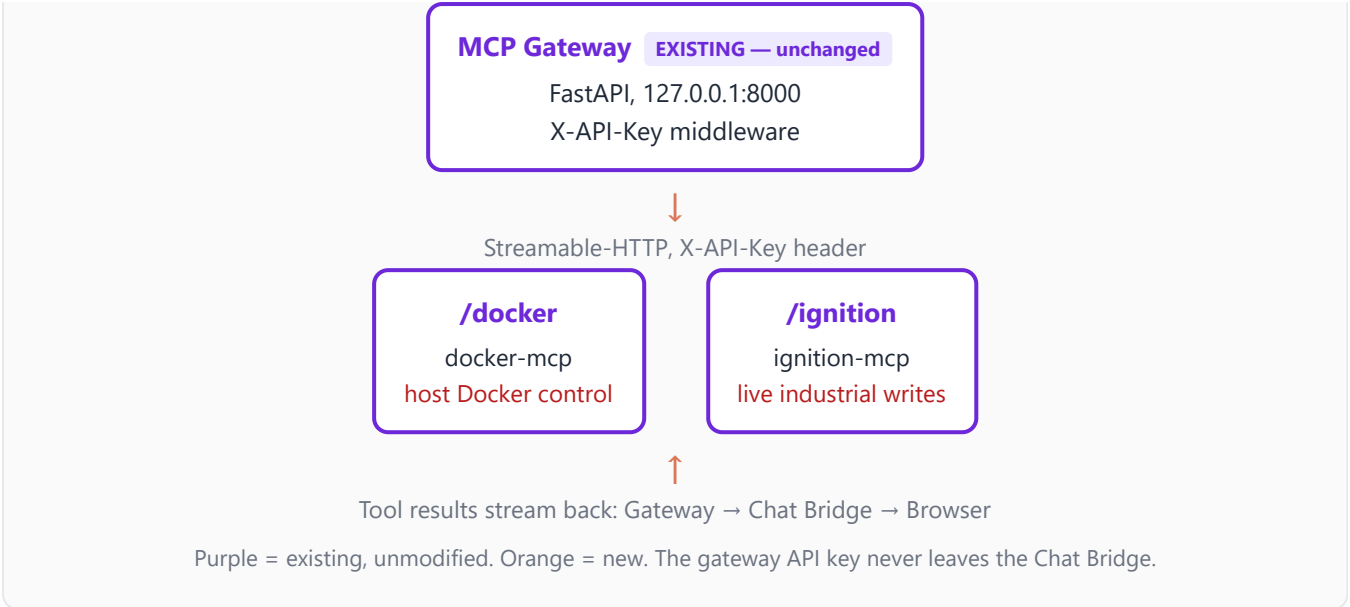
Key decision: build the Chat Bridge in **Python/FastAPI** (matching the existing gateway's stack) using the **Claude Agent SDK's Python package**, `claude-agent-sdk`, rather than introducing a second language/runtime (e.g. Node.js) purely for this layer. The Bridge acts as an MCP *client* of the gateway's existing endpoints — it does not reimplement Docker or Ignition access, it just configures the SDK's `mcpServers` to point at the gateway's own `/docker` and `/ignition` URLs, carrying the `X-API-Key` header as a service-held secret:

```
mcp_servers = {
    "docker": {"type": "http", "url": "http://127.0.0.1:8000/docker", "headers": {"X-API-Key": GATEWAY_API_KEY}},
    "ignition": {"type": "http", "url": "http://127.0.0.1:8000/ignition", "headers": {"X-API-Key": GATEWAY_API_KEY}},
}
```

(Exact SDK config keys/transport name to confirm against the installed `claude-agent-sdk` version before implementation.)

4. System Flow Diagram





5. Component Breakdown

Component	Status	Responsibility	Notes
Angular frontend	New	Chat UI: input box, message list, streaming render, tool-activity indicators	Signals for message state; render markdown/code; show distinct state while a Docker/Ignition tool call is in flight
Chat Bridge (FastAPI, Python)	New	Authenticates browser users, runs <code>claude-agent-sdk query()</code> per conversation, streams <code>SDKMessage</code> events to the client, holds <code>GATEWAY_API_KEY</code>	Sibling service to <code>mcp-gateway</code> , same stack/conventions (pydantic-settings, uvicorn); use raw-ASGI streaming like the gateway already does, not <code>BaseHTTPMiddleware</code>
MCP Gateway	Existing — unchanged	Exposes <code>/docker</code> and <code>/ignition</code> as Streamable-HTTP MCP servers	Keep bound to <code>127.0.0.1</code> or an internal network only; the Chat Bridge is the only intended caller
docker-mcp / ignition-mcp	Existing — unchanged	Actual tool implementations (Docker socket, Ignition WebDev endpoints)	No changes needed

6. Step-by-Step Implementation Plan

- Capture the current Claude Code → Gateway contract.** Find the `.mcp.json` / `~/.claude.json` entries Claude Code currently uses to reach `http://127.0.0.1:8000/docker` and `/ignition` (transport type, headers) so the Chat Bridge reproduces the exact same connection.
- Scaffold the Chat Bridge as a sibling service.** New directory alongside `mcp-gateway/` (e.g. `chat-bridge/`), FastAPI + uvicorn, pydantic-settings config following the existing `GATEWAY_`-prefixed pattern (e.g. `BRIDGE_` prefix). Add `claude-agent-sdk` and `ANTHROPIC_API_KEY` as a Bridge-only secret.

3. **Point the SDK at the existing gateway.** Configure `mcpServers` in the Bridge to call the gateway's own `/docker` and `/ignition` URLs with the `X-API-Key` header, using `GATEWAY_API_KEY` loaded server-side. Do not duplicate or bypass the gateway's tool logic.
4. **Build the streaming `/chat` endpoint.** WebSocket route on the Bridge, implemented so streaming isn't buffered (same constraint the gateway's own middleware was written around) — forward each `SDKMessage` (text delta, tool-call start/result, final result) to the browser as it arrives.
5. **Add session continuity.** Capture `session_id` per browser conversation and pass it back via the SDK's `resume` option on the next `query()` call.
6. **Add real end-user authentication on the Bridge.** This is the "real auth layer" the gateway's own comments call for. Individual login/session per browser user, separate from and in addition to the Bridge→Gateway `X-API-Key`. Do not let unauthenticated browser traffic reach the Bridge's `query()` call at all.
7. **Add tool-risk guardrails.** Because `/docker` can create/mount containers (host root via the Docker socket) and `/ignition` can write to a live industrial system, decide whether certain tool calls need explicit user confirmation in the UI, scoped permissions per user, or an allow-list, before wiring the browser path live.
8. **Build the Angular chat UI.** Message list + input, a service opening the WebSocket and exposing streamed tokens as signals, a visibly distinct state for "running a Docker/Ignition tool call" vs. plain text.
9. **Wire frontend to Bridge to Gateway end-to-end locally.** Run `mcp-gateway` and the new Bridge together (both on loopback during dev); verify a full round trip including a real Docker and a real Ignition tool call.
10. **Test edge cases.** Gateway unreachable/restarted mid-session, invalid/rotated `GATEWAY_API_KEY`, long-running Docker/Ignition calls, dropped WebSocket + reconnect, concurrent browser sessions.
11. **Deploy.** Extend the existing `docker-compose.yml` with a second service for the Chat Bridge on the same internal network as `gateway`; keep the gateway's port **not** published externally (only the Bridge is internet/reverse-proxy facing); serve the Angular build behind the same or a separate reverse proxy with WebSocket upgrade support and TLS.

7. Key Technical Considerations

- **Trust boundary moves, doesn't disappear:** today the trust boundary is "whoever can reach the terminal running Claude Code." After this change it's "whoever can authenticate to the Chat Bridge." That boundary must be at least as strong, given `/docker` and `/ignition` both allow high-impact actions.
- **Two different secrets, two different lifetimes:** `GATEWAY_API_KEY` (Bridge→Gateway, service credential) and end-user auth (Browser→Bridge, human credential) should be managed and rotated independently.
- **Streaming implementation:** follow the same pattern the gateway already uses — avoid response-buffering middleware on any streaming route, in the Bridge as well as the Gateway.

- **Network exposure:** only the Chat Bridge should ever be reachable from outside the host/internal network; `mcp-gateway` stays on `127.0.0.1` or an internal Docker network, exactly as its own docker-compose comments already recommend.
- **No changes to `mcp-gateway`, `docker-mcp`, or `ignition-mcp` are required** for this integration — the Bridge is purely additive.

8. Quick Reference

Item	Value
Existing gateway location	<code>C:\Users\katlego.gagoopane\Documents\2026\Claude\backend\mcp-gateway</code>
Existing gateway endpoints	<code>GET /health</code> (open), <code>/docker</code> , <code>/ignition</code> (require <code>X-API-Key</code>)
Existing gateway default bind	<code>127.0.0.1:8000</code> (host dev), <code>0.0.0.0:8000</code> in compose
New service	Chat Bridge — FastAPI (Python), sibling to <code>mcp-gateway</code>
Python SDK package	<code>claude-agent-sdk</code>
Credential (new)	<code>ANTHROPIC_API_KEY</code> on the Chat Bridge only
Credential (existing, reused)	<code>GATEWAY_API_KEY</code> , sent by the Bridge as <code>X-API-Key</code> to the Gateway
Session continuity	<code>resume: session_id</code> option on <code>query()</code> , captured from the prior result event

9. Open Questions to Resolve Before Building

- What end-user auth mechanism should the Chat Bridge use (SSO, simple login, VPN + trusted network only)?
- Should any Docker or Ignition tool calls require explicit in-UI confirmation before executing, given their real-world impact?
- Should conversation history persist across browser sessions, or is in-memory-per-connection acceptable initially?
- Where will the Chat Bridge and Gateway be hosted relative to each other (same host/network, as assumed here, or split further)?
- Expected user base: a small internal/operations team, or broader access?

v2 — revised to build on the existing FastAPI MCP Gateway (`docker-mcp` + `ignition-mcp`) rather than a generic/hypothetical backend. See v1 (`mcp_frontend.pdf`) for the original generic-architecture draft.